Dueling Network Architectures for Deep Reinforcement Learning

Ziyu Wang ZIYU@GOOGLE.COM
Tom Schaul SCHAUL@GOOGLE.COM
Matteo Hessel MTTHSS@GOOGLE.COM
Hado van Hasselt HADO@GOOGLE.COM
Marc Lanctot LANCTOT@GOOGLE.COM
Nando de Freitas NANDODEFREITAS@GOOGLE.COM
Google DeepMind, London, UK

Abstract

In recent years there have been many successes of using deep representations in reinforcement learning. Still, many of these applications use conventional architectures, such as convolutional networks, LSTMs, or auto-encoders. In this paper, we present a new neural network architecture for model-free reinforcement learning. Our dueling network represents two separate estimators: one for the state value function and one for the state-dependent action advantage function. The main benefit of this factoring is to generalize learning across actions without imposing any change to the underlying reinforcement learning algorithm. Our results show that this architecture leads to better policy evaluation in the presence of many similar-valued actions. Moreover, the dueling architecture enables our RL agent to outperform the state-of-the-art on the Atari 2600 domain.

1. Introduction

Over the past years, deep learning has contributed to dramatic advances in scalability and performance of machine learning (LeCun et al., 2015). One exciting application is the sequential decision-making setting of reinforcement learning (RL) and control. Notable examples include deep Q-learning (Mnih et al., 2015), deep visuomotor policies (Levine et al., 2015), attention with recurrent networks (Ba et al., 2015), and model predictive control with embeddings (Watter et al., 2015). Other recent successes include massively parallel frameworks (Nair et al., 2015) and expert move prediction in the game of Go (Maddison et al., 2015), which produced policies matching those of Monte Carlo tree search programs, and squarely beaten a professional player when combined with search (Silver et al., 2016).

In spite of this, most of the approaches for RL use standard neural networks, such as convolutional networks, MLPs, LSTMs and autoencoders. The focus in these recent advances has been on designing improved control and RL algorithms, or simply on incorporating existing neural network architectures into RL methods. Here, we take an *alternative but complementary approach* of focusing primarily on innovating a neural network architecture that is better suited for model-free RL. This approach has the benefit that the new network can be easily combined with existing and future algorithms for RL. That is, this paper advances a new network (Figure 1), but uses already published algorithms.

The proposed network architecture, which we name the *dueling architecture*, explicitly separates the representation of

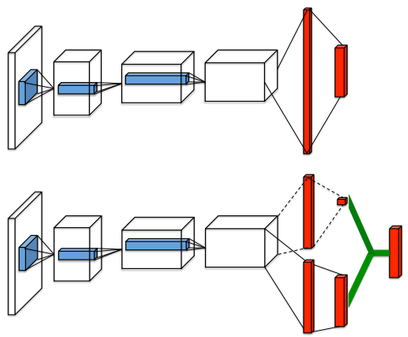


Figure 1. A popular single stream Q-network (**top**) and the dueling Q-network (**bottom**). The dueling network has two streams to separately estimate (scalar) state-value and the advantages for each action; the green output module implements equation (9) to combine them. Both networks output Q-values for each action.

Proceedings of the 33rd International Conference on Machine Learning, New York, NY, USA, 2016. JMLR: W&CP volume 48. Copyright 2016 by the author(s).

Due用于深度强化学习的 ling 网络架构

Ziyu Wang ZIYU@GOOGLE.COM
Tom Schaul SCHAUL@GOOGLE.COM
Matteo Hessel MTTHSS@GOOGLE.COM
哈多·范哈塞尔特 HADO@GOOGLE.COM
马克·兰克托 LANCTOT@GOOGLE.COM
南多·德弗雷塔斯 NANDODEFREITAS@GOOGLE.COM
Google DeepMind，英国伦敦

摘要

近年来，在强化学习中使用深度表示已取得诸多成功。然而，其中许多应用仍采用传统架构，例如卷积网络、LSTMs 或自编码器。本文提出了一种用于无模型强化学习的新型神经网络架构。我们的对偶网络包含两个独立的估计器：一个用于状态价值函数，另一个用于依赖于状态的动作优势函数。这种分解的主要优势在于，无需对底层强化学习算法作任何改动，即可使学习在不同动作之间泛化。我们的结果表明，在存在许多价值相近的动作时，该架构能够实现更好的策略评估。此外，对偶架构使我们的强化学习智能体在 Atari 2600 领域超越了当前最先进的方法。

1. 引言

过去几年，深度学习推动了机器学习在可扩展性和性能方面的显著进步 (LeCun et al., 2015)。一项令人振奋的应用是强化学习 (RL) 和控制中的序贯决策场景。典型示例包括深度 Q 学习 (Mnih et al., 2015)、深度视觉运动策略 (Levine et al., 2015)、结合循环网络的注意力机制 (Ba et al., 2015)，以及结合嵌入表示的模型预测控制 (Watter et al., 2015)。近期的其他成功案例包括大规模并行框架 (Nair et al., 2015) 和围棋中的专家落子预测 (Maddison et al., 2015)，后者生成的策略可与蒙特卡洛树搜索程序的策略相媲美，并在与搜索相结合时，以明显优势击败了一名职业棋手 (Silveret al.,2016)。

尽管如此，大多数 RL 方法仍使用标准神经网络，例如卷积网络、MLPs、LSTMs 和自编码器。这些近期进展的重点在于设计改进的控制与 RL 算法，或仅仅将现有神经网络架构融入 RL 方法。在此，我们采用一种不同但互补的方法，主要致力于创新一种更适合无模型 RL 的神经网络架构。这种方法的优势在于，新网络可以轻松地与现有及未来的 RL 算法结合。也就是说，本文提出了一种新网络（图 1），但使用的是已发表的算法。

所提出的网络架构被我们命名为对偶架构，它显式地分离了以下两者的表示：

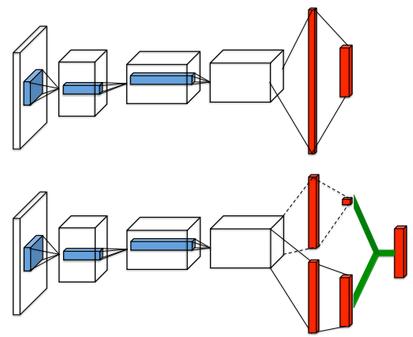


图 *1.*一种常用的单流 Q 网络（上）和对偶 Q 网络（下）。对偶网络具有两个流，分别估计（标量）状态价值和每个动作的优势；绿色输出模块实现公式 (9)，将二者结合。两个网络均针对每个动作 output Q-values。

会议论文集： 33rd 国际机器学习会议，美国纽约州纽约市，2016。JMLR: W&CP 第 48 卷。 版权归作者所有，2016。

state values and (state-dependent) action advantages. The dueling architecture consists of two streams that represent the value and advantage functions, while sharing a common convolutional feature learning module. The two streams are combined via a special aggregating layer to produce an estimate of the state-action value function Q as shown in Figure 1. This dueling network should be understood as a single Q network with two streams that replaces the popular single-stream Q network in existing algorithms such as Deep Q-Networks (DQN; Mnih et al., 2015). The dueling network automatically produces separate estimates of the state value function and advantage function, without any extra supervision.

Intuitively, the dueling architecture can learn which states are (or are not) valuable, without having to learn the effect of each action for each state. This is particularly useful in states where its actions do not affect the environment in any relevant way. To illustrate this, consider the saliency maps shown in Figure 2[1]. These maps were generated by computing the Jacobians of the trained value and advantage streams with respect to the input video, following the method proposed by Simonyan et al. (2013). (The experimental section describes this methodology in more detail.) The figure shows the value and advantage saliency maps for two different time steps. In one time step (leftmost pair of images), we see that the value network stream pays attention to the road and in particular to the horizon, where new cars appear. It also pays attention to the score. The advantage stream on the other hand does not pay much attention to the visual input because its action choice is practically irrelevant when there are no cars in front. However, in the second time step (rightmost pair of images) the advantage stream pays attention as there is a car immediately in front, making its choice of action very relevant.

In the experiments, we demonstrate that the dueling architecture can more quickly identify the correct action during policy evaluation as redundant or similar actions are added to the learning problem.

We also evaluate the gains brought in by the dueling architecture on the challenging Atari 2600 testbed. Here, an RL agent with the same structure and hyper-parameters must be able to play 57 different games by observing image pixels and game scores only. The results illustrate vast improvements over the single-stream baselines of Mnih et al. (2015) and van Hasselt et al. (2015). The combination of prioritized replay (Schaul et al., 2016) with the proposed dueling network results in the new state-of-the-art for this popular domain.

[1]https://www.youtube.com/playlist?list=PLVFXyCSfS2Pau0gBh0mwTxDmutywWyFBP

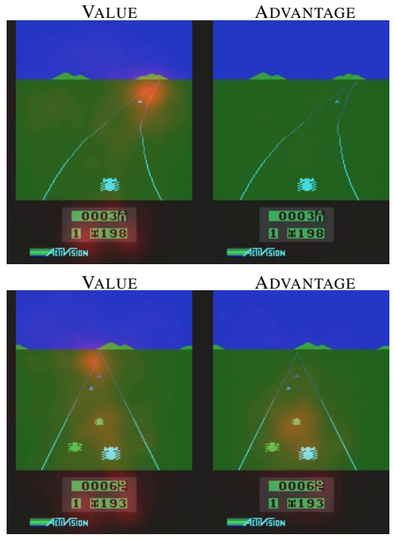


Figure 2. See, attend and drive: Value and advantage saliency maps (red-tinted overlay) on the Atari game Enduro, for a trained dueling architecture. The value stream learns to pay attention to the road. The advantage stream learns to pay attention only when there are cars immediately in front, so as to avoid collisions.

1.1. Related Work

The notion of maintaining separate value and advantage functions goes back to Baird (1993). In Baird's original advantage updating algorithm, the shared Bellman residual update equation is decomposed into two updates: one for a state value function, and one for its associated advantage function. Advantage updating was shown to converge faster than Q-learning in simple continuous time domains in (Harmon et al., 1995). Its successor, the advantage learning algorithm, represents only a single advantage function (Harmon & Baird, 1996).

The dueling architecture represents both the value $V(s)$ and advantage $A(s,a)$ functions with a single deep model whose output combines the two to produce a state-action value $Q(s,a)$. Unlike in advantage updating, the representation and algorithm are decoupled by construction. Consequently, the dueling architecture can be used in combination with a myriad of model free RL algorithms.

There is a long history of advantage functions in policy gradients, starting with (Sutton et al., 2000). As a recent example of this line of work, Schulman et al. (2015) estimate advantage values online to reduce the variance of policy gradient algorithms.

There have been several attempts at playing Atari with deep

状态价值和（依赖于状态的）动作优势。对偶架构由两个分别表示价值函数和优势函数的分支组成，同时共享一个卷积特征学习模块。这两个分支通过一个特殊的聚合层进行组合，以生成状态-动作价值函数 Q 的估计，如图 1 所示。应将这一对偶网络理解为一个具有两个分支的单一 Q 网络，它替代了现有算法（例如深度 Q 网络（DQN；Mnih et al., 2015））中广泛使用的单分支 Q 网络。对偶网络无需任何额外监督，即可自动生成状态价值函数和优势函数的独立估计。

直观而言，对偶架构能够学习哪些状态有价值（或没有价值），而无需学习每个状态下每个动作的影响。这在动作不会对环境产生任何相关影响的状态下尤为有用。为说明这一点，请看图 2[1] 中的显著性图。这些图遵循 Simonyan 等人（2013）提出的方法，通过计算训练后的价值流和优势流相对于输入视频的雅可比矩阵生成。（实验部分更详细地介绍了这一方法。）该图展示了两个不同时刻的价值显著性图和优势显著性图。在其中一个时刻（最左侧的一对图像），我们可以看到，价值网络流关注道路，尤其是新车辆出现的地平线。它还关注得分。另一方面，优势流对视觉输入并不太关注，因为当前方没有车辆时，其动作选择实际上无关紧要。然而，在第二个时刻（最右侧的一对图像），由于正前方有一辆车，动作选择变得非常重要，因此优势流会对此予以关注。

在实验中，我们证明，随着学习问题中加入冗余或相似的动作，竞争架构能够在策略评估过程中更快地识别正确的动作。

我们还在具有挑战性的 Atari 2600 测试平台上评估了对偶架构带来的收益。在这里，具有相同结构和超参数的强化学习智能体必须仅通过观察图像像素和游戏得分，就能玩 57 种不同的游戏。结果表明，与 Mnih et al. (2015) 和 van Hasselt et al. (2015) 的单流基线相比，该架构取得了大幅改进。将优先经验回放（Schaul et al., 2016）与所提出的对偶网络相结合，在这一热门领域取得了新的最先进性能。

[1]https://www.youtube.com/playlist?list=PLVFXyCSfS2Pau0gBh0mwTxDmutywWyFBP

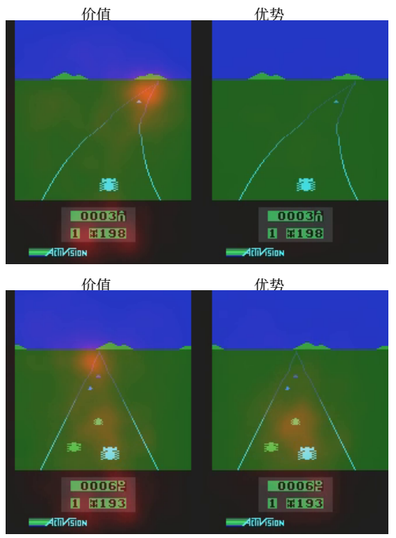


图 2.观察、关注并驾驶：训练后的对偶架构在 Atari 游戏 Enduro 中的价值和优势显著性图（红色叠加层）。价值流学会关注道路。优势流学会仅在正前方有车辆时予以关注，以避免碰撞。

1.1. 相关工作

分别维护价值函数和优势函数的概念可以追溯到 Baird (1993)。在 Baird 最初的优势更新算法中，共享的贝尔曼残差更新方程被分解为两个更新：一个用于状态价值函数，另一个用于与之关联的优势函数。Harmon et al. (1995) 表明，在简单的连续时间领域中，优势更新比 Q-learning 收敛得更快。其后继算法，即优势学习算法，仅表示一个优势函数（Harmon & Baird, 1996）。

对偶架构使用单个深度模型同时表示价值函数 $V(s)$ 和优势函数 $A(s,a)$，其输出将二者结合，生成状态-动作价值 $Q(s,a)$。与优势更新不同，该架构在设计上将表示与算法解耦。因此，对偶架构可以与多种无模型强化学习算法结合使用。

优势函数在策略梯度中的应用历史悠久，始于 (Sutton et al., 2000)。作为这一研究方向的近期示例，Schulman et al. (2015) 在线估计优势值，以降低策略梯度算法的方差。

已有若干尝试使用深度

reinforcement learning, including Mnih et al. (2015); Guo et al. (2014); Stadie et al. (2015); Nair et al. (2015); van Hasselt et al. (2015); Bellemare et al. (2016) and Schaul et al. (2016). The results of Schaul et al. (2016) are the current published state-of-the-art.

2. Background

We consider a sequential decision making setup, in which an agent interacts with an environment $\mathcal{E}$ over discrete time steps, see Sutton & Barto (1998) for an introduction. In the Atari domain, for example, the agent perceives a video s_t consisting of M image frames: $s_t = (x_{t-M+1}, \ldots, x_t) \in \mathcal{S}$ at time step t. The agent then chooses an action from a discrete set $a_t \in \mathcal{A} = \{1, \ldots, |\mathcal{A}|\}$ and observes a reward signal r_t produced by the game emulator.

The agent seeks to maximize the expected discounted return, where we define the discounted return as $R_t = \sum_{\tau=t}^{\infty} \gamma^{\tau-t} r_\tau$. In this formulation, $\gamma \in [0, 1]$ is a discount factor that trades-off the importance of immediate and future rewards.

For an agent behaving according to a stochastic policy π, the values of the state-action pair (s, a) and the state s are defined as follows

$$\begin{aligned} Q^\pi(s,a) &= \mathbb{E}\left[R_t \middle| s_t = s, a_t = a, \pi\right], \text{ and} \\ V^\pi(s) &= \mathbb{E}_{a\sim\pi(s)}\left[Q^\pi(s,a)\right]. \end{aligned} \tag{1}$$

The preceding state-action value function (Q function for short) can be computed recursively with dynamic programming:

$$Q^\pi(s,a) = \mathbb{E}_{s'}\left[r + \gamma \mathbb{E}_{a'\sim\pi(s')}\left[Q^\pi(s',a')\right] \mid s, a, \pi\right].$$

We define the optimal $Q^*(s,a) = \max_\pi Q^\pi(s,a)$. Under the deterministic policy $a = \arg\max_{a'\in\mathcal{A}} Q^*(s,a')$, it follows that $V^*(s) = \max_a Q^*(s,a)$. From this, it also follows that the optimal Q function satisfies the Bellman equation:

$$Q^*(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^*(s',a') \mid s, a\right]. \tag{2}$$

We define another important quantity, the *advantage function*, relating the value and Q functions:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s). \tag{3}$$

Note that $\mathbb{E}_{a\sim\pi(s)}\left[A^\pi(s,a)\right] = 0$. Intuitively, the value function V measures how good it is to be in a particular state s. The Q function, however, measures the value of choosing a particular action when in this state. The advantage function subtracts the state value from Q to obtain a relative measure of the importance of each action.

2.1. Deep Q-networks

The value functions as described in the preceding section are high dimensional objects. To approximate them, we can use a deep Q-network: $Q(s,a;\theta)$ with parameters θ. To estimate this network, we optimize the following sequence of loss functions at iteration i:

$$L_i(\theta_i) = \mathbb{E}_{s,a,r,s'}\left[\left(y_i^{DQN} - Q(s,a;\theta_i)\right)^2\right], \tag{4}$$

with

$$y_i^{DQN} = r + \gamma \max_{a'} Q(s',a';\theta^-), \tag{5}$$

where θ^- represents the parameters of a fixed and separate *target network*. We could attempt to use standard Q-learning to learn the parameters of the network $Q(s,a;\theta)$ online. However, this estimator performs poorly in practice. A key innovation in (Mnih et al., 2015) was to freeze the parameters of the target network $Q(s',a';\theta^-)$ for a fixed number of iterations while updating the *online network* $Q(s,a;\theta_i)$ by gradient descent. (This greatly improves the stability of the algorithm.) The specific gradient update is

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a,r,s'}\left[\left(y_i^{DQN} - Q(s,a;\theta_i)\right)\nabla_{\theta_i} Q(s,a;\theta_i)\right]$$

This approach is model free in the sense that the states and rewards are produced by the environment. It is also off-policy because these states and rewards are obtained with a behavior policy (epsilon greedy in DQN) different from the online policy that is being learned.

Another key ingredient behind the success of DQN is *experience replay* (Lin, 1993; Mnih et al., 2015). During learning, the agent accumulates a dataset $\mathcal{D}_t = \{e_1, e_2, \ldots, e_t\}$ of experiences $e_t = (s_t, a_t, r_t, s_{t+1})$ from many episodes. When training the Q-network, instead only using the current experience as prescribed by standard temporal-difference learning, the network is trained by sampling mini-batches of experiences from $\mathcal{D}$ uniformly at random. The sequence of losses thus takes the form

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{U}(\mathcal{D})}\left[\left(y_i^{DQN} - Q(s,a;\theta_i)\right)^2\right].$$

Experience replay increases data efficiency through re-use of experience samples in multiple updates and, importantly, it reduces variance as uniform sampling from the replay buffer reduces the correlation among the samples used in the update.

2.2. Double Deep Q-networks

The previous section described the main components of DQN as presented in (Mnih et al., 2015). In this paper,

强化学习玩 Atari 游戏，包括 Mnih 等人 (2015)；Guo 等人 (2014)；Stadie 等人 (2015)；Nair 等人 (2015)；van Hasselt 等人 (2015)；Bellemare 等人 (2016) 和 Schaul 等人 (2016)。Schaul 等人 (2016) 的结果是目前已发表研究中的最先进水平。

2. 背景

我们考虑一种序贯决策设定，其中智能体在离散时间步上与环境 $\mathcal{E}$ 交互，相关介绍参见 Sutton & Barto (1998)。例如，在 Atari 领域中，智能体在时间步 t感知由 M个图像帧组成的视频 s_t：$s_t = (x_{t-M+1}, \ldots, x_t) \in \mathcal{S}$。随后，智能体从离散集合 $a_t \in \mathcal{A} = \{1, \ldots, |\mathcal{A}|\}$ 中选择一个动作，并观测游戏模拟器给出的奖励 signal r_t produced。

智能体力求最大化期望折扣回报，其中我们将折扣回报定义为 $R_t = \sum_{\tau=t}^{\infty} \gamma^{\tau-t} r_\tau$。在这一表述中，$\gamma \in [0, 1]$ 是一个折扣因子，用于权衡即时奖励与未来奖励的重要性。

对于按照随机策略 policy π行动的智能体，状态-动作对 (s, a) 的价值和状态 s 的价值定义如下

$$\begin{aligned} Q^\pi(s,a) &= \mathbb{E}\left[R_t \middle| s_t = s, a_t = a, \pi\right], \text{ and} \\ V^\pi(s) &= \mathbb{E}_{a\sim\pi(s)}\left[Q^\pi(s,a)\right]. \end{aligned} \tag{1}$$

上述状态-动作价值函数（简称Q 函数）可通过动态规划递归计算：

$$Q^\pi(s,a) = \mathbb{E}_{s'}\left[r + \gamma \mathbb{E}_{a'\sim\pi(s')}\left[Q^\pi(s',a')\right] \mid s, a, \pi\right].$$

我们定义最优的 $Q^*(s,a) = \max_\pi Q^\pi(s,a)$。在确定性策略 $a = \arg\max_{a'\in\mathcal{A}} Q^*(s,a')$下，可得 $V^*(s) = \max_a Q^*(s,a)$。由此还可得，最优 Q 函数满足贝尔曼方程：

$$Q^*(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^*(s',a') \mid s, a\right]. \tag{2}$$

我们定义另一个重要的量，即优势函数，它关联价值 and Q functions：

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s). \tag{3}$$

注意，$\mathbb{E}_{a\sim\pi(s)}\left[A^\pi(s,a)\right] = 0$。直观地说，价值函数 V 衡量处于特定状态 s 的好坏程度。然而，Q 函数衡量在该状态下选择某个特定动作的价值。优势函数减去状态价值 from Q to，得到衡量各个动作重要性的相对指标。

2.1. 深度Q网络

上一节所述的价值函数是高维对象。为了近似这些函数，我们可以使用一个深度 Q 网络：$Q(s,a;\theta)$，其参数为 θ。为了估计该网络，我们在 iteration i 处优化以下损失函数序列：

$$L_i(\theta_i) = \mathbb{E}_{s,a,r,s'}\left[\left(y_i^{DQN} - Q(s,a;\theta_i)\right)^2\right], \tag{4}$$

其中

$$y_i^{DQN} = r + \gamma \max_{a'} Q(s',a';\theta^-), \tag{5}$$

其中，θ^- 表示一个固定且独立的目标网络的参数。我们可以尝试使用标准的 Q-学习来在线学习网络 $Q(s,a;\theta)$的参数。然而，这种估计器在实践中表现不佳。(Mnih et al., 2015)中的一项关键创新是，在固定次数的迭代中冻结目标网络 $Q(s',a';\theta^-)$的参数，同时通过梯度下降更新在线网络 $Q(s,a;\theta_i)$。（这极大地提高了算法的稳定性。）具体的梯度更新为

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a,r,s'}\left[\left(y_i^{DQN} - Q(s,a;\theta_i)\right)\nabla_{\theta_i} Q(s,a;\theta_i)\right]$$

这种方法是无模型的，因为状态和奖励由环境产生。它也是离策略的，因为这些状态和奖励是通过一种行为策略（在 DQN 中为 epsilon 贪心策略）获得的，该行为策略不同于正在学习的在线策略。

DQN 成功的另一个关键要素是经验回放（Lin, 1993; Mnih et al., 2015 ）。在学习过程中，智能体从多个回合中积累由经验 $e_t = (s_t, a_t, r_t, s_{t+1})$ 构成的数据集 $\mathcal{D}_t = \{e_1, e_2, \ldots, e_t\}$。训练 Q-网络时，并非仅使用标准时序差分学习所规定的当前经验，而是从 $\mathcal{D}$ 中均匀随机采样经验的小批量来训练网络。因此，损失序列具有如下形式

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{U}(\mathcal{D})}\left[\left(y_i^{DQN} - Q(s,a;\theta_i)\right)^2\right].$$

经验回放通过在多次更新中重复使用经验样本来提高数据效率，而且更为重要的是，它还降低了方差，因为从回放缓冲区中均匀采样降低了用于更新的样本之间的相关性。

2.2. 双重深度 Q 网络

上一节介绍了 (Mnih et al., 2015) 中提出的 DQN 的主要组成部分。在本文中，

we use the improved Double DQN (DDQN) learning algorithm of van Hasselt et al. (2015). In Q-learning and DQN, the max operator uses the same values to both select and evaluate an action. This can therefore lead to overoptimistic value estimates (van Hasselt, 2010). To mitigate this problem, DDQN uses the following target:

$$y_i^{DDQN} = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta_i); \theta^-). \quad (6)$$

DDQN is the same as for DQN (see Mnih et al. (2015)), but with the target y_i^{DQN} replaced by y_i^{DDQN}. The pseudo-code for DDQN is presented in Appendix A.

2.3. Prioritized Replay

A recent innovation in prioritized experience replay (Schaul et al., 2016) built on top of DDQN and further improved the state-of-the-art. Their key idea was to increase the replay probability of experience tuples that have a high expected learning progress (as measured via the proxy of absolute TD-error). This led to both faster learning and to better final policy quality across most games of the Atari benchmark suite, as compared to uniform experience replay.

To strengthen the claim that our dueling architecture is complementary to algorithmic innovations, we show that it improves performance for both the uniform and the prioritized replay baselines (for which we picked the easier to implement rank-based variant), with the resulting prioritized dueling variant holding the new state-of-the-art.

3. The Dueling Network Architecture

The key insight behind our new architecture, as illustrated in Figure 2, is that for many states, it is unnecessary to estimate the value of each action choice. For example, in the Enduro game setting, knowing whether to move left or right only matters when a collision is imminent. In some states, it is of paramount importance to know which action to take, but in many other states the choice of action has no repercussion on what happens. For bootstrapping based algorithms, however, the estimation of state values is of great importance for every state.

To bring this insight to fruition, we design a single Q-network architecture, as illustrated in Figure 1, which we refer to as the dueling network. The lower layers of the dueling network are convolutional as in the original DQNs (Mnih et al., 2015). However, instead of following the convolutional layers with a single sequence of fully connected layers, we instead use two sequences (or streams) of fully connected layers. The streams are constructed such that they have they have the capability of providing separate estimates of the value and advantage functions. Finally, the two streams are combined to produce a single output Q function. As in (Mnih et al., 2015), the output of the network is a set of Q values, one for each action.

Since the output of the dueling network is a Q function, it can be trained with the many existing algorithms, such as DDQN and SARSA. In addition, it can take advantage of any improvements to these algorithms, including better replay memories, better exploration policies, intrinsic motivation, and so on.

The module that combines the two streams of fully-connected layers to output a Q estimate requires very thoughtful design.

From the expressions for advantage $Q^\pi(s,a) = V^\pi(s) + A^\pi(s,a)$ and state-value $V^\pi(s) = \mathbb{E}_{a\sim\pi(s)}[Q^\pi(s,a)]$, it follows that $\mathbb{E}_{a\sim\pi(s)}[A^\pi(s,a)] = 0$. Moreover, for a deterministic policy, $a^* = \arg\max_{a'\in\mathcal{A}} Q(s,a')$, it follows that $Q(s,a^*) = V(s)$ and hence $A(s,a^*) = 0$.

Let us consider the dueling network shown in Figure 1, where we make one stream of fully-connected layers output a scalar $V(s;\theta,\beta)$, and the other stream output an $|\mathcal{A}|$-dimensional vector $A(s,a;\theta,\alpha)$. Here, θ denotes the parameters of the convolutional layers, while α and β are the parameters of the two streams of fully-connected layers.

Using the definition of advantage, we might be tempted to construct the aggregating module as follows:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + A(s,a;\theta,\alpha), \quad (7)$$

Note that this expression applies to all (s,a) instances; that is, to express equation (7) in matrix form we need to replicate the scalar, $V(s;\theta,\beta)$, $|\mathcal{A}|$ times.

However, we need to keep in mind that $Q(s,a;\theta,\alpha,\beta)$ is only a parameterized estimate of the true Q-function. Moreover, it would be wrong to conclude that $V(s;\theta,\beta)$ is a good estimator of the state-value function, or likewise that $A(s,a;\theta,\alpha)$ provides a reasonable estimate of the advantage function.

Equation (7) is unidentifiable in the sense that given Q we cannot recover V and A uniquely. To see this, add a constant to $V(s;\theta,\beta)$ and subtract the same constant from $A(s,a;\theta,\alpha)$. This constant cancels out resulting in the same Q value. This lack of identifiability is mirrored by poor practical performance when this equation is used directly.

To address this issue of identifiability, we can force the advantage function estimator to have zero advantage at the chosen action. That is, we let the last module of the network implement the forward mapping

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \max_{a'\in|\mathcal{A}|} A(s,a';\theta,\alpha)\right). \quad (8)$$

我们采用 van Hasselt et al. (2015) 提出的改进型 Double DQN (DDQN) 学习算法。在 Q-learning 和 DQN 中，max 算子使用相同的值来选择和评估动作。因此，这可能导致过于乐观的价值估计 (van Hasselt,2010)。为缓解这一问题，DDQN 使用以下目标：

$$y_i^{DDQN} = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta_i); \theta^-). \quad (6)$$

DDQN 与 DQN 相同（参见 Mnih 等人 (2015)），但将目标 y_i^{DQN} 替换为 y_i^{DDQN}。DDQN 的伪代码见附录 A。

2.3. 优先经验回放

近期，优先经验回放方面的一项创新（Schaul 等人，2016）建立在 DDQN 的基础上，进一步提升了当前最佳性能。其核心思想是提高预期学习进展较大的经验元组的回放概率（以 TD 误差的绝对值作为代理指标来衡量）。与均匀经验回放相比，这使得在 Atari 基准测试集的大多数游戏中，学习速度更快，最终策略质量也更高。

为进一步支持我们的对偶架构与算法创新具有互补性这一论断，我们展示了该架构能够提升均匀回放和优先回放两种基线的性能（对于优先回放，我们选择了更易实现的基于排序的变体），由此得到的优先回放对偶变体取得了新的最佳性能。

3. 对偶网络架构

如图 2 所示，我们的新架构背后的关键洞见是：对于许多状态，无需估计每个可选动作的价值。例如，在 Enduro 游戏场景中，只有当碰撞即将发生时，判断应该向左还是向右移动才有意义。在某些状态下，知道应采取哪个动作至关重要，但在许多其他状态下，动作的选择并不会影响后续发生的情况。然而，对于基于自举的算法，每个状态的状态价值估计都非常重要。

为实现这一思路，我们设计了一个单一的 Q- 网络架构，如图 1 所示，我们将其称为对偶网络。对偶网络的底层与原始 DQN（Mnih et al., 2015）一样，采用卷积层。然而，我们并未在卷积层之后接入单一的全连接层序列，而是使用两个全连接层序列（或流）。这两个流的构造使其能够分别估计价值函数和优势函数。最后，将这两个流组合起来，生成单一的输出 Q 函数。与（Mnih et al., 2015）中一样，网络的输出是一个集合 of Q values，其中每个动作对应一个元素。

由于对偶网络的输出是一个 Q 函数，因此可以使用许多现有算法进行训练，例如 DDQN 和 SARSA。此外，它还可以受益于这些算法的任何改进，包括更好的经验回放存储、更好的探索策略、内在动机等。

将两路全连接层合并以输出 a Q estimate 的模块需要经过十分周密的设计。

由优势函数 $Q^\pi(s,a) = V^\pi(s) + A^\pi(s,a)$ 和状态价值函数 $V^\pi(s) = \mathbb{E}_{a\sim\pi(s)}[Q^\pi(s,a)]$, 的表达式可得 $\mathbb{E}_{a\sim\pi(s)}[A^\pi(s,a)] = 0$。此外，对于确定性策略， $a^* = \arg\max_{a'\in\mathcal{A}} Q(s,a')$, 可得 $Q(s,a^*) = V(s)$，因此 $A(s,a^*) = 0$。

考虑图 1 所示的对偶网络，其中我们使一个全连接层分支输出标量 $V(s;\theta,\beta)$，另一个分支输出 $|\mathcal{A}|$ 维向量 $A(s,a;\theta,\alpha)$。这里， θ 表示卷积层的参数，while α and β 是两个全连接层分支的参数。

根据优势的定义，我们可能会倾向于按如下方式构建聚合模块：

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + A(s,a;\theta,\alpha), \quad (7)$$

注意，此表达式适用于所有 (s,a) 实例；也就是说，要以矩阵形式表示方程 (7)，我们需要将标量 $V(s;\theta,\beta)$ 复制 $|\mathcal{A}|$ 次。

然而，我们需要牢记， $Q(s,a;\theta,\alpha,\beta)$ 只是真实 Q 函数的参数化估计。此外，若据此断定 $V(s;\theta,\beta)$ 是状态价值函数的良好估计器，或同样断定 $A(s,a;\theta,\alpha)$ 能够提供对优势函数的合理估计，都是错误的。

方程 (7) 是不可辨识的，即给定 Q，我们无法唯一地 recover V and A 。为说明这一点，可给 $V(s;\theta,\beta)$ 加上一个常数，并从 from$A(s,a;\theta,\alpha)$ 中减去同一个常数。该常数相互抵消，从而得到相同的 Q 值。直接使用此方程时，实际性能较差，这也反映了这种不可辨识性。

为解决这一可辨识性问题，我们可以强制优势函数估计器在所选动作处的优势为零。也就是说，我们让网络的最后一个模块实现如下前向映射

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \max_{a'\in|\mathcal{A}|} A(s,a';\theta,\alpha)\right). \quad (8)$$

Now, for $a^* = \arg\max_{a' \in \mathcal{A}} Q(s, a'; \theta, \alpha, \beta) = \arg\max_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha)$, we obtain $Q(s, a^*; \theta, \alpha, \beta) = V(s; \theta, \beta)$. Hence, the stream $V(s; \theta, \beta)$ provides an estimate of the value function, while the other stream produces an estimate of the advantage function.

An alternative module replaces the max operator with an average:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right). \quad (9)$$

On the one hand this loses the original semantics of V and A because they are now off-target by a constant, but on the other hand it increases the stability of the optimization: with (9) the advantages only need to change as fast as the mean, instead of having to compensate any change to the optimal action's advantage in (8). We also experimented with a softmax version of equation (8), but found it to deliver similar results to the simpler module of equation (9). Hence, all the experiments reported in this paper use the module of equation (9).

Note that while subtracting the mean in equation (9) helps with identifiability, it does not change the relative rank of the A (and hence Q) values, preserving any greedy or ϵ-greedy policy based on Q values from equation (7). When acting, it suffices to evaluate the advantage stream to make decisions.

It is important to note that equation (9) is viewed and implemented as part of the network and not as a separate algorithmic step. Training of the dueling architectures, as with standard Q networks (e.g. the deep Q-network of Mnih et al. (2015)), requires only back-propagation. The estimates $V(s; \theta, \beta)$ and $A(s, a; \theta, \alpha)$ are computed automatically without any extra supervision or algorithmic modifications.

As the dueling architecture shares the same input-output interface with standard Q networks, we can recycle all learning algorithms with Q networks (*e.g.*, DDQN and SARSA) to train the dueling architecture.

4. Experiments

We now show the practical performance of the dueling network. We start with a simple policy evaluation task and then show larger scale results for learning policies for general Atari game-playing.

4.1. Policy evaluation

We start by measuring the performance of the dueling architecture on a policy evaluation task. We choose this particular task because it is very useful for evaluating network architectures, as it is devoid of confounding factors such as the choice of exploration strategy, and the interaction between policy improvement and policy evaluation.

In this experiment, we employ temporal difference learning (without eligibility traces, *i.e.*, $\lambda = 0$) to learn Q values. More specifically, given a behavior policy π, we seek to estimate the state-action value $Q^\pi(\cdot, \cdot)$ by optimizing the sequence of costs of equation (4), with target

$$y_i = r + \gamma \mathbb{E}_{a' \sim \pi(s')} \left[Q(s', a'; \theta_i) \right].$$

The above update rule is the same as that of Expected SARSA (van Seijen et al., 2009). We, however, do not modify the behavior policy as in Expected SARSA.

To evaluate the learned Q values, we choose a simple environment where the exact $Q^\pi(s, a)$ values can be computed separately for all $(s, a) \in \mathcal{S} \times \mathcal{A}$. This environment, which we call the *corridor* is composed of three connected corridors. A schematic drawing of the corridor environment is shown in Figure 3, The agent starts from the bottom left corner of the environment and must move to the top right to get the largest reward. A total of 5 actions are available: go up, down, left, right and no-op. We also have the freedom of adding an arbitrary number of no-op actions. In our setup, the two vertical sections both have 10 states while the horizontal section has 50.

We use an ϵ-greedy policy as the behavior policy π, which chooses a random action with probability ϵ or an action according to the optimal Q function $\arg\max_{a \in \mathcal{A}} Q^*(s, a)$ with probability $1 - \epsilon$. In our experiments, ϵ is chosen to be 0.001.

We compare a single-stream Q architecture with the dueling architecture on three variants of the corridor environment with 5, 10 and 20 actions respectively. The 10 and 20 action variants are formed by adding no-ops to the original environment. We measure performance by Squared Error (SE) against the true state values: $\sum_{s \in \mathcal{S}, a \in \mathcal{A}} (Q(s, a; \theta) - Q^\pi(s, a))^2$. The single-stream architecture is a three layer MLP with 50 units on each hidden layer. The dueling architecture is also composed of three layers. After the first hidden layer of 50 units, however, the network branches off into two streams each of them a two layer MLP with 25 hidden units. The results are summarized in Figure 3.

The results show that with 5 actions, both architectures converge at about the same speed. However, when we increase the number of actions, the dueling architecture performs better than the traditional Q-network. In the dueling network, the stream $V(s; \theta, \beta)$ learns a general value that is shared across many similar actions at s, hence leading to faster convergence. This is a very promising result because many control tasks with large action spaces have this prop-

现在，对于 $a^* = \arg\max_{a' \in \mathcal{A}} Q(s, a'; \theta, \alpha, \beta) = \arg\max_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha)$，我们得到 $Q(s, a^*; \theta, \alpha, \beta) = V(s; \theta, \beta)$。因此，分支 $V(s; \theta, \beta)$ 提供价值函数的估计，而另一分支则产生优势函数的估计。

另一种模块用平均值替代最大值算子：

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right). \quad (9)$$

一方面，这会使 $V \text{and} A$ because 失去原有的语义，因为它们现在与目标值相差一个常数；但另一方面，这提高了优化的稳定性：在式 (9) 中，优势只需以与均值相同的速度变化，而不必补偿式 (8) 中最优动作优势的任何变化。我们还尝试了式 (8) 的 softmax 版本，但发现其结果与式 (9) 中更简单的模块相近。因此，本文报告的所有实验均采用式 (9) 中的模块。

请注意，虽然在式 (9) 中减去均值有助于保证可辨识性，但这不会改变 A （因而也不会改变 Q）值的相对排序，从而保留了基于式 (7) 中 Q 值的任何贪婪策略或 ϵ-贪婪策略。在执行动作时，只需计算优势分支即可作出决策。

需要注意的是，方程 (9) 被视为网络的一部分并作为网络的一部分实现，而非单独的算法步骤。与标准 Q 网络（例如 Mnih et al. (2015) 的深度 Q网络）一样，决斗架构的训练仅需反向传播。估计值 $V(s; \theta, \beta)$ 和 $A(s, a; \theta, \alpha)$ 自动计算，无需任何额外监督或算法修改。

由于决斗架构与标准 Q 网络具有相同的输入输出接口，我们可以复用所有用于 Q 网络的学习算法（例如，DDQN 和 SARSA）来训练决斗架构。

4. 实验

现在我们展示决斗网络的实际性能。我们从一个简单的策略评估任务开始，然后展示在更大规模上学习通用 Atari 游戏策略的结果。

4.1. 策略评估

我们首先衡量决斗架构在策略评估任务上的性能。我们选择这一特定的任务，是因为它非常适合用于评估网络架构，因为其中不存在探索策略的选择以及策略改进与策略评估之间的交互等混杂因素。

在本实验中，我们采用时序差分学习（不使用资格迹，即，$\lambda = 0$）来 learn Q values. 更具体地说，给定行为策略 π，我们通过优化方程 (4) 中的代价序列来估计状态-动作价值 $Q^\pi(\cdot, \cdot)$，其目标为

$$y_i = r + \gamma \mathbb{E}_{a' \sim \pi(s')} \left[Q(s', a'; \theta_i) \right].$$

上述更新规则与 Expected SARSA（van Seijen et al., 2009）的更新规则相同。然而，我们并不像 Expected SARSA 那样修改行为策略。

为了评估学习到的 Q 值，我们选择了一个简单的环境，在该环境中，可以针对所有 $(s, a) \in \mathcal{S} \times \mathcal{A}$ 分别计算精确的 $Q^\pi(s, a)$ 值。我们将这个环境称为走廊，它由三条相互连接的走廊组成。走廊环境的示意图如图 3 所示。智能体从环境的左下角出发，必须移动到右上角才能获得最大奖励。共有 5 种可用动作：向上、向下、向左、向右以及空操作。我们还可以自由添加任意数量的空操作动作。在我们的设置中，两个竖直部分各有 10 个状态，而水平部分有 50 个状态。

我们使用 ϵ-贪心策略作为行为策略 π，该策略以 probability ϵ or 选择随机动作，以概率 $1 - \epsilon$ 根据最优 Q 函数 $\arg\max_{a \in \mathcal{A}} Q^*(s, a)$ 选择动作。在我们的实验中，ϵ 取值为 0.001。

我们在走廊环境的三种变体上比较单流 Q 架构与对偶架构，这三种变体分别有 5、10 和 20 个动作。具有 10 和 20 个动作的变体通过向原始环境添加空操作构建而成。我们通过相对于真实状态值的平方误差 (SE) 衡量性能：$\sum_{s \in \mathcal{S}, a \in \mathcal{A}} (Q(s, a; \theta) - Q^\pi(s, a))^2$。单流架构是一个三层 MLP，每个隐藏层有 50 个单元。对偶架构也由三层组成。不过，在具有 50 个单元的第一个隐藏层之后，网络分为两个流，每个流都是一个具有 25 个隐藏单元的两层 MLP。结果汇总于图 3。

结果表明，当有 5 个动作时，两种架构的收敛速度大致相同。然而，当我们增加动作数量时，决斗架构的表现优于传统的 Q-网络。在决斗网络中，分支 $V(s; \theta, \beta)$ 学习一个在 s 处由许多相似动作共享的通用价值，从而加快收敛。这一结果非常有前景，因为许多具有大动作空间的控制任务都具有这一特-

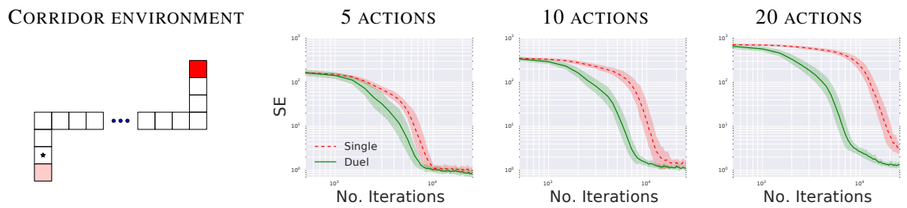


Figure 3. **(a)** The corridor environment. The star marks the starting state. The redness of a state signifies the reward the agent receives upon arrival. The game terminates upon reaching either reward state. The agent's actions are going up, down, left, right and no action. Plots **(b), (c)** and **(d)** shows squared error for policy evaluation with 5, 10, and 20 actions on a log-log scale. The dueling network (Duel) consistently outperforms a conventional single-stream network (Single), with the performance gap increasing with the number of actions.

erty, and consequently we should expect that the dueling network to lead to much faster convergence than a traditional single stream network. In the following section, we will indeed see that the dueling network results in substantial gains in performance in a wide-range of Atari games.

4.2. General Atari Game-Playing

We perform a comprehensive evaluation of our proposed method on the Arcade Learning Environment (Bellemare et al., 2013), which is composed of 57 Atari games. The challenge is to deploy a single algorithm and architecture, with a fixed set of hyper-parameters, to learn to play all the games given only raw pixel observations and game rewards. This environment is very demanding because it is both comprised of a large number of highly diverse games and the observations are high-dimensional.

We follow closely the setup of van Hasselt et al. (2015) and compare to their results using single-stream Q-networks. We train the dueling network with the DDQN algorithm (see Appendix A). At the end of this section, we incorporate prioritized experience replay (Schaul et al., 2016).

Our network architecture has the same low-level convolutional structure of DQN (Mnih et al., 2015; van Hasselt et al., 2015). There are 3 convolutional layers followed by 2 fully-connected layers. The first convolutional layer has 32 8×8 filters with stride 4, the second 64 4×4 filters with stride 2, and the third and final convolutional layer 64 3×3 filters with stride 1. As shown in Figure 1, the dueling network splits into two streams of fully connected layers. The value and advantage streams both have a fully-connected layer with 512 units. The final hidden layers of the value and advantage streams are both fully-connected with the value stream having one output and the advantage as many outputs as there are valid actions[2]. We combine the value and advantage streams using the module described by Equation (9). Rectifier non-linearities (Fukushima, 1980) are inserted between all adjacent layers.

We adopt the optimizers and hyper-parameters of van Hasselt et al. (2015), with the exception of the learning rate which we chose to be slightly lower (we do not do this for double DQN as it can deteriorate its performance). Since both the advantage and the value stream propagate gradients to the last convolutional layer in the backward pass, we rescale the combined gradient entering the last convolutional layer by $1/\sqrt{2}$. This simple heuristic mildly increases stability. In addition, we clip the gradients to have their norm less than or equal to 10. This clipping is not standard practice in deep RL, but common in recurrent network training (Bengio et al., 2013).

To isolate the contributions of the dueling architecture, we re-train DDQN with a single stream network using exactly the same procedure as described above. Specifically, we apply gradient clipping, and use 1024 hidden units for the first fully-connected layer of the network so that both architectures (dueling and single) have roughly the same number of parameters. We refer to this re-trained model as *Single Clip*, while the original trained model of van Hasselt et al. (2015) is referred to as *Single*.

As in (van Hasselt et al., 2015), we start the game with up to *30 no-op* actions to provide random starting positions for the agent. To evaluate our approach, we measure improvement in percentage (positive or negative) in score over the better of human and baseline agent scores:

$$\frac{\text{Score}_{\text{Agent}} - \text{Score}_{\text{Baseline}}}{\max\{\text{Score}_{\text{Human}}, \text{Score}_{\text{Baseline}}\} - \text{Score}_{\text{Random}}}. \quad (10)$$

We took the maximum over human and baseline agent scores as it prevents insignificant changes to appear as large improvements when neither the agent in question nor the baseline are doing well. For example, an agent that achieves 2% human performance should not be interpreted as being twice as good when the baseline agent achieves 1% human performance. We also chose not to measure

[2]The number of actions ranges between 3-18 actions in the ALE environment.

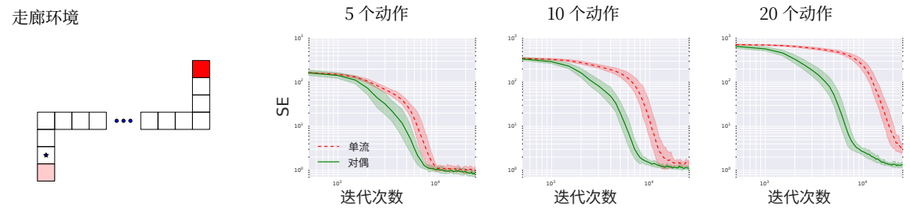


图 *3.***(a)** 走廊环境。星号标记起始状态。状态的红色深浅表示智能体到达该状态时获得的奖励。到达任一奖励状态时，游戏结束。智能体的动作包括向上、向下、向左、向右和不采取动作。图 **(b), (c)** 和 **(d)** 以双对数坐标展示了在 5、10 和 20 个动作下进行策略评估的平方误差。对偶网络（Duel）的性能始终优于传统的单流网络（Single），且性能差距随动作数量的增加而扩大。

性，因此我们应当预期，决斗网络的收敛速度会远快于传统的单分支网络。在下一节中，我们确实将看到，决斗网络在多种 Atari 游戏中带来了显著的性能提升。

4.2. 通用 Atari 游戏玩法

我们在由 57 款 Atari 游戏组成的街机学习环境（Arcade Learning Environment，Bellemare et al., 2013）中，对所提出的方法进行了全面评估。其挑战在于，采用单一算法和架构以及一组固定的超参数，仅凭原始像素观测和游戏奖励来学习玩所有游戏。这一环境极具挑战性，因为它既包含大量差异显著的游戏，又具有高维观测。

我们严格遵循 van Hasselt et al. (2015) 的实验设置，并与其使用单流 Q 网络所得的结果进行比较。我们使用 DDQN 算法训练对偶网络（见附录 A）。在本节末尾，我们引入优先经验回放（Schaul et al., 2016）。

我们的网络架构具有与 DQN（Mnih et al., 2015; van Hasselt et al., 2015）相同的底层卷积结构。共有 3 个卷积层，后接 2 个全连接层。第一个卷积层 has 32 8×8 filters，步幅为 4；第二个卷积层有 64 4×4 个滤波器，步幅为 2；第三个也是最后一个卷积层 layer64 3×3 filters，步幅为 1。如图 1 所示，决斗网络分为两个由全连接层构成的分支。价值分支和优势分支各有一个包含 512 个单元的全连接层。价值分支和优势分支的最后一个隐藏层均为全连接层，其中价值分支有一个输出，优势分支的输出数量与有效的 actions[2]. 的数量相同。我们使用以下公式所描述的模块来组合价值分支和优势分支：

[2]在 ALE 环境中，动作数量范围为 3-18。

公式 (9)。所有相邻层之间均插入了整流非线性函数（Fukushima, 1980）。

我们采用 van Hasselt et al. (2015) 的优化器和超参数，但将学习率设得略低一些（对于双重 DQN，我们不作此调整，因为这可能降低其性能）。由于优势流和价值流在反向传播过程中都会将梯度传递到最后一个卷积层，我们将传入最后一个卷积层的合并梯度乘以 $1/\sqrt{2}$ 进行缩放。这一简单的启发式方法略微提高了稳定性。此外，我们对梯度进行裁剪，使其范数小于或等于 10。这种裁剪并非深度强化学习中的标准做法，但在循环网络训练中很常见（Bengio et al., 2013）。

为了单独考察对偶架构的贡献，我们采用与上述完全相同的流程，使用单流网络重新训练 DDQN。具体而言，我们应用梯度裁剪，并在网络的第一个全连接层中使用 1024 个隐藏单元，使两种架构（对偶架构和单流架构）的参数数量大致相同。我们将这个重新训练的模型称为 单流裁剪，而将 van Hasselt 等人（2015）原先训练的模型称为单流。

与（van Hasselt 等人，2015）中的做法一样，我们在游戏开始时执行最多 *30* 个无操作 动作，为智能体提供随机的起始位置。为了评估我们的方法，我们以人类得分和基准智能体得分中较高者为基准，衡量得分提升的百分比（正值或负值）：

$$\frac{\text{Score}_{\text{Agent}} - \text{Score}_{\text{Baseline}}}{\max\{\text{Score}_{\text{Human}}, \text{Score}_{\text{Baseline}}\} - \text{Score}_{\text{Random}}}. \quad (10)$$

我们取人类得分与基线智能体得分中的最大值，因为当所考察的智能体和基线智能体表现都不佳时，这样可以避免将微小的变化呈现为大幅提升。例如，当基线智能体达到人类表现的 1% 时，不应将达到人类表现的 2% 的智能体解读为表现好了一倍。我们还选择不衡量

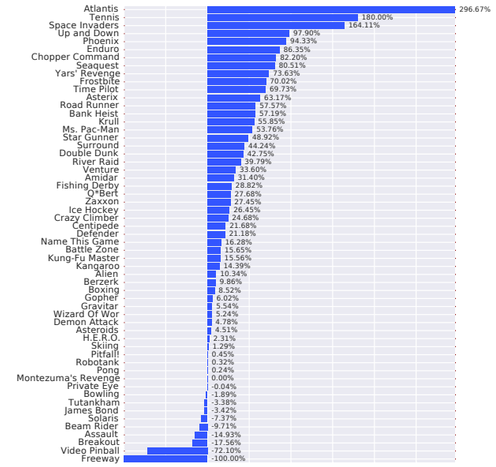


Figure 4. Improvements of dueling architecture over the baseline Single network of van Hasselt et al. (2015), using the metric described in Equation (10). Bars to the right indicate by how much the dueling network outperforms the single-stream network.

performance in terms of percentage of human performance alone because a tiny difference relative to the baseline on some games can translate into hundreds of percent in human performance difference.

The results for the wide suite of 57 games are summarized in Table 1. Detailed results are presented in the Appendix.

Using this 30 no-ops performance measure, the dueling network (Duel Clip) does substantially better than the Single Clip network of similar capacity. It also does considerably better than the baseline (Single) of van Hasselt et al. (2015). For reference, we also show results for the deep Q-network of Mnih et al. (2015), referred to as Nature DQN.

Figure 4 shows the improvement of the dueling network over the baseline Single network of van Hasselt et al. (2015). Again, we seen that the improvements are often very dramatic.

As shown in Table 1, Single Clip performs better than Single. We verified that this gain was mostly brought in by gradient clipping. For this reason, we incorporate gradient clipping in all the new approaches.

Duel Clip does better than Single Clip on 75.4% of the games (43 out of 57). It also achieves higher scores compared to the Single baseline on 80.7% (46 out of 57) of the games. Of all the games with 18 actions, Duel Clip is better 86.6% of the time (26 out of 30). This is consistent with the findings of the previous section. Overall, our agent (Duel Clip) achieves human level performance on 42 out of 57

Table 1. Mean and median scores across all 57 Atari games, measured in percentages of human performance.

	30 no-ops		Human Starts	
	Mean	**Median**	**Mean**	**Median**
Prior. Duel Clip	**591.9%**	**172.1%**	**567.0%**	**115.3%**
Prior. Single	434.6%	123.7%	386.7%	112.9%
Duel Clip	**373.1%**	**151.5%**	**343.8%**	**117.1%**
Single Clip	341.2%	132.6%	302.8%	114.1%
Single	307.3%	117.8%	332.9%	110.9%
Nature DQN	227.9%	79.1%	219.6%	68.5%

games. Raw scores for all the games, as well as measurements in human performance percentage, are presented in the Appendix.

Robustness to human starts. One shortcoming of the 30 no-ops metric is that an agent does not necessarily have to generalize well to play the Atari games. Due to the deterministic nature of the Atari environment, from an unique starting point, an agent could learn to achieve good performance by simply remembering sequences of actions.

To obtain a more robust measure, we adopt the methodology of Nair et al. (2015). Specifically, for each game, we use 100 starting points sampled from a human expert's trajectory. From each of these points, an evaluation episode is launched for up to 108,000 frames. The agents are evaluated only on rewards accrued after the starting point. We refer to this metric as *Human Starts*.

As shown in Table 1, under the Human Starts metric, Duel Clip once again outperforms the single stream variants. In particular, our agent does better than the Single baseline on 70.2% (40 out of 57) games and on games of 18 actions, Duel Clip is 83.3% better (25 out of 30).

Combining with Prioritized Experience Replay. The dueling architecture can be easily combined with other algorithmic improvements. In particular, prioritization of the experience replay has been shown to significantly improve performance of Atari games (Schaul et al., 2016). Furthermore, as prioritization and the dueling architecture address very different aspects of the learning process, their combination is promising. So in our final experiment, we investigate the integration of the dueling architecture with prioritized experience replay. We use the prioritized variant of DDQN (Prior. Single) as the new baseline algorithm, which replaces with the uniform sampling of the experience tuples by rank-based prioritized sampling. We keep all the parameters of the prioritized replay as described in (Schaul et al., 2016), namely a priority exponent of 0.7, and an annealing schedule on the importance sampling exponent from 0.5 to 1. We combine this baseline with our dueling architecture (as above), and again use gradient clipping (Prior. Duel Clip).

Note that, although orthogonal in their objectives, these

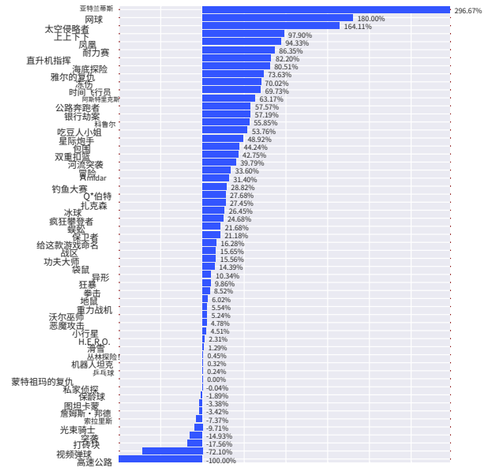


图 *4.* 对偶架构相对于 van Hasselt et al. (2015) 的基线 Single 网络的改进，采用公式 (10) 中描述的指标。向右延伸的条形表示对偶网络优于单流网络的幅度。

仅以人类表现的百分比表示的性能，因为在某些游戏中，相对于基线的微小差异可能转化为以人类表现百分比衡量的数百个百分点的差异。

57 款游戏的广泛测试结果汇总于表1。详细结果见附录。

采用这一 30 次空操作性能度量，对偶网络 (Duel Clip) 的表现显著优于容量相近的 Single Clip 网络。它的表现也明显优于 van Hasselt et al. (2015) 的基线 (Single)。作为参考，我们还展示了 Mnih et al. (2015) 的深度 Q-网络的结果，该网络称为 Nature DQN。

图 4 展示了对偶网络相对于 van Hasselt et al. (2015) 的基线 Single 网络的改进。我们再次看到，这些改进往往非常显著。

如表 1 所示，Single Clip 的表现优于 Single。我们验证了这一提升主要来自梯度裁剪。因此，我们在所有新方法中都采用梯度裁剪。

Duel Clip 在 75.4% 的游戏（57 款中的 43 款）中表现优于 Single Clip。与 Single 基线相比，它还在 80.7% 的游戏（57 款中的 46 款）中取得了更高的分数。在所有具有 18 个动作的游戏中，Duel Clip 在 86.6% 的游戏（30 款中的 26 款）中表现更好。这与上一节的发现一致。总体而言，我们的智能体（Duel Clip）在 57 款游戏中的 42 款中达到了人类水平的表现

表 *1.* 全部 57 款 Atari 游戏的平均得分和中位数得分，以人类表现的百分比衡量。

	30 次空操作		人类起始状态	
	平均值	中位数	均值	中位数
优先化 双流 裁剪	**591.9%**	**172.1%**	**567.0%**	**115.3%**
优先单网络	434.6%	123.7%	386.7%	112.9%
对决 裁剪	**373.1%**	**151.5%**	**343.8%**	**117.1%**
单流 裁剪	341.2%	132.6%	302.8%	114.1%
单一	307.3%	117.8%	332.9%	110.9%
《自然》DQN	227.9%	79.1%	219.6%	68.5%

。所有游戏的原始分数以及以人类表现百分比衡量的结果均列于附录中。

对人类起始状态的鲁棒性。30 次无操作指标的一个不足是，智能体未必需要具备良好的泛化能力就能玩 Atari 游戏。由于 Atari 环境具有确定性，从一个固定的起点出发，智能体只需记住动作序列，就可能学会取得良好的表现。

为获得更稳健的衡量指标，我们采用 Nair et al. (2015) 的方法。具体而言，对于每款游戏，我们从人类专家的轨迹中采样 100 个起点。从每个起点启动一个评估回合，最多运行 108,000 帧。评估智能体时，仅计入起点之后累积的奖励。我们将这一指标称为人类起始状态。

如表 1 所示，在人类起始状态指标下，Duel Clip 再次优于单流变体。具体而言，我们的智能体在 70.2% 的游戏（57 款中的 40 款）上优于 Single 基线；在具有 18 个动作的游戏中，Duel Clip 在 83.3% 的游戏（30 款中的 25 款）上表现更好。

与优先经验回放相结合。决斗架构可以很容易地与其他算法改进相结合。特别是，已有研究表明，对经验回放进行优先级排序可显著提升 Atari 游戏的性能 (Schaul et al., 2016)。此外，由于优先级排序与决斗架构针对学习过程中截然不同的方面，二者的结合很有前景。因此，在最后一项实验中，我们研究了决斗架构与优先经验回放的整合。我们使用 DDQN 的优先级变体 (Prior. Single) 作为新的基线算法，用基于排名的优先采样替代经验元组的均匀采样。我们保留 (Schaul et al., 2016) 中所述的优先回放的所有参数，即优先级指数为 0.7，重要性采样指数按退火计划从 0.5 增至 1。我们将此基线与我们的决斗架构相结合（如上所述），并再次使用梯度裁剪 (Prior. Duel Clip)。

请注意，尽管这些扩展在目标上相互独立，但它们

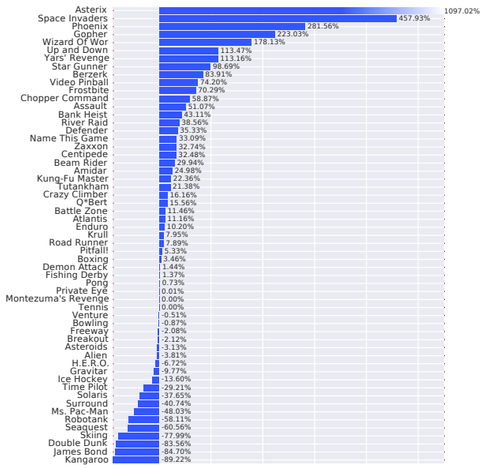


Figure 5. Improvements of dueling architecture over Prioritized DDQN baseline, using the same metric as Figure 4. Again, the dueling architecture leads to significant improvements over the single-stream baseline on the majority of games.

extensions (prioritization, dueling and gradient clipping) interact in subtle ways. For example, prioritization interacts with gradient clipping, as sampling transitions with high absolute TD-errors more often leads to gradients with higher norms. To avoid adverse interactions, we roughly re-tuned the learning rate and the gradient clipping norm on a subset of 9 games. As a result of rough tuning, we settled on 6.25×10^{-5} for the learning rate and 10 for the gradient clipping norm (the same as in the previous section).

When evaluated on all 57 Atari games, our prioritized dueling agent performs significantly better than both the prioritized baseline agent and the dueling agent alone. The full mean and median performance against the human performance percentage is shown in Table 1. When initializing the games using up to 30 no-ops action, we observe mean and median scores of 591% and 172% respectively. The direct comparison between the prioritized baseline and prioritized dueling versions, using the metric described in Equation 10, is presented in Figure 5.

The combination of prioritized replay and the dueling network results in vast improvements over the previous state-of-the-art in the popular ALE benchmark.

Saliency maps. To better understand the roles of the value and the advantage streams, we compute saliency maps (Simonyan et al., 2013). More specifically, to visualize the salient part of the image as seen by the value stream, we compute the absolute value of the Jacobian of $\widehat{V}$ with respect to the input frames: $|\nabla_s \widehat{V}(s;\theta)|$. Similarly, to visualize the salient part of the image as seen by the advantage stream, we compute $|\nabla_s \widehat{A}(s, \arg\max_{a'} \widehat{A}(s,a');\theta)|$. Both quantities are of the same dimensionality as the input frames and therefore can be visualized easily alongside.

Here, we place the gray scale input frames in the green and blue channel and the saliency maps in the red channel. All three channels together form an RGB image. Figure 2 depicts the value and advantage saliency maps on the Enduro game for two different time steps. As observed in the introduction, the value stream pays attention to the horizon where the appearance of a car could affect future performance. The value stream also pays attention to the score. The advantage stream, on the other hand, cares more about cars that are on an immediate collision course.

5. Discussion

The advantage of the dueling architecture lies partly in its ability to learn the state-value function efficiently. With every update of the Q values in the dueling architecture, the value stream V is updated – this contrasts with the updates in a single-stream architecture where only the value of one actions is updated, the values for all other actions remain untouched. This more frequent updating of the value stream in our approach allocates more resources to V, and thus allows for better approximation of the state values, which need to be accurate for temporal-difference-based methods like Q-learning to work (Sutton & Barto, 1998). This phenomenon is reflected in the experiments, where the advantage of the dueling architecture over single-stream Q networks grows when the number of actions is large.

Furthermore, the differences between Q-values for a given state are often very small relative to the magnitude of Q. For example, after training with DDQN on the game of Seaquest, the average action gap (the gap between the Q values of the best and the second best action in a given state) across visited states is roughly 0.04, whereas the average state value across those states is about 15. This difference in scale means that small amount of noise in the updates could reorder the actions, and thus making the nearly greedy policy switch abruptly. The dueling architecture with its separate advantage stream is robust to such effects.

6. Conclusions

We introduced a new neural network architecture that decouples value and advantage in deep Q-networks, while sharing a common feature learning module. The new dueling architecture, in combination with some algorithmic improvements, leads to dramatic improvements over existing approaches for deep RL in the challenging Atari domain. The results presented in this paper are the new state-of-the-art in this popular domain.

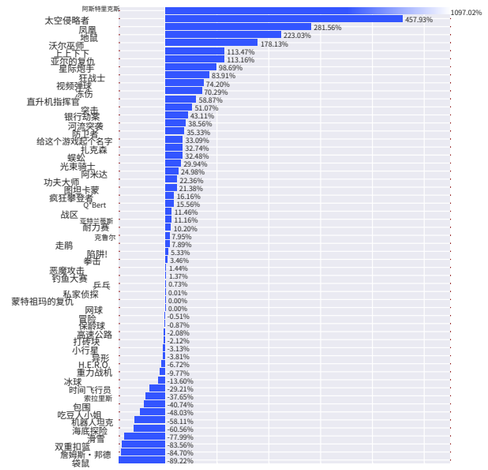


图 5. 对偶架构相较于优先级 DDQN 基线的性能提升，采用与图 4 相同的度量指标。同样，对偶架构在大多数游戏中相较于单流基线取得了显著提升。

（优先级采样、对偶架构和梯度裁剪）之间存在微妙的相互作用。例如，优先级采样会与梯度裁剪相互作用，因为更频繁地采样绝对 TD 误差较大的转移会导致梯度范数增大。为避免不利的相互作用，我们在由 9 个游戏组成的子集上对学习率和梯度裁剪范数进行了粗略的重新调优。经过粗略调优，我们最终将学习率设为 6.25×10^{-5}，将梯度裁剪范数设为 10（与上一节相同）。

在全部 57 款 Atari 游戏上进行评估时，我们的优先级对偶智能体的表现显著优于优先级基线智能体和单独使用的对偶智能体。表 1 展示了以人类表现百分比衡量的完整平均性能和中位数性能。在使用最多 30 次无操作动作初始化游戏时，我们观察到平均得分和中位数得分分别为 591% 和 172%。图 5 使用公式 10 中描述的度量指标，展示了优先级基线版本与优先级对偶版本的直接比较。

优先级经验回放与对偶网络的结合，在广泛使用的 ALE 基准上相较于此前的最先进方法取得了大幅提升。

显著性图。为了更好地理解价值流和优势流的作用，我们计算显著性图（Simonyan et al., 2013）。更具体地说，为了可视化价值流所看到的图像显著部分，we compute V 相对于输入帧的雅可比矩阵的绝对值：$|\nabla_s V(s;\theta)|$. 类似地，为了可视化优势流所看到的图像显著部分，我们计算 $|\nabla_s A(s,\arg\max_{a'} A(s,a');\theta)|$。这两个量的维度均与输入帧相同，因此可以方便地并排可视化。

这里，我们将灰度输入帧放入绿色和蓝色通道，将显著性图放入红色通道。三个通道共同构成一幅 RGB 图像。图 2 展示了 Enduro 游戏中两个不同时刻的价值显著性图和优势显著性图。如引言中所述，价值分支关注地平线，因为那里出现的汽车可能影响未来的表现。价值分支也关注得分。另一方面，优势分支更关注即将发生碰撞的汽车。

5. 讨论

对偶架构的优势部分在于它能够高效地学习状态价值函数。对偶架构中的 Q 值每次更新时，价值分支 V 都会更新——这与单分支架构中的更新不同，后者仅更新一个动作的价值，而所有其他动作的价值均保持不变。我们的方法更频繁地更新价值分支，为 V分配了更多资源，从而能够更好地逼近状态价值；对于 Q-learning 等基于时序差分的方法，准确的状态价值是其有效运行的必要条件（Sutton & Barto, 1998）。这一现象在实验中得到了体现：当动作数量较大时，对偶架构相对于单分支 Q网络的优势随之增大。

此外，对于给定状态，Q值之间的差异相对于 Q的量级通常很小。例如，在 Seaquest 游戏上使用 DDQN 训练后，在访问过的状态中，平均动作间隙（给定状态下最优动作与次优动作的 Q值之差）约为 0.04，而这些状态的平均状态价值约为 15。这种尺度差异意味着，更新中的少量噪声就可能改变动作的排序，从而使近似贪心策略发生突变。决斗架构具有独立的优势分支，对此类影响具有鲁棒性。

6. 结论

我们提出了一种新的神经网络架构，在共享同一个特征学习模块的同时，将深度 Q-网络中的价值与优势解耦。这种新的对偶架构结合若干算法改进，在具有挑战性的 Atari 领域中，相较于现有深度强化学习方法实现了显著的性能提升。本文所呈现的结果在这一热门领域达到了新的最先进水平。

References

Ba, J., Mnih, V., and Kavukcuoglu, K. Multiple object recognition with visual attention. In *ICLR*, 2015.

Baird, L.C. Advantage updating. Technical Report WL-TR-93-1146, Wright-Patterson Air Force Base, 1993.

Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

Bellemare, M. G., Ostrovski, G., Guez, A., Thomas, P. S., and Munos, R. Increasing the action gap: New operators for reinforcement learning. In *AAAI*, 2016. To appear.

Bengio, Y., Boulanger-Lewandowski, N., and Pascanu, R. Advances in optimizing recurrent networks. In *ICASSP*, pp. 8624–8628, 2013.

Fukushima, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36:193–202, 1980.

Guo, X., Singh, S., Lee, H., Lewis, R. L., and Wang, X. Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning. In *NIPS*, pp. 3338–3346. 2014.

Harmon, M.E. and Baird, L.C. Multi-player residual advantage learning with general function approximation. Technical Report WL-TR-1065, Wright-Patterson Air Force Base, 1996.

Harmon, M.E., Baird, L.C., and Klopf, A.H. Advantage updating applied to a differential game. In G. Tesauro, D.S. Touretzky and Leen, T.K. (eds.), *NIPS*, 1995.

LeCun, Y., Bengio, Y., and Hinton, G. Deep learning. *Nature*, 521(7553):436–444, 2015.

Levine, S., Finn, C., Darrell, T., and Abbeel, P. End-to-end training of deep visuomotor policies. *arXiv preprint arXiv:1504.00702*, 2015.

Lin, L.J. *Reinforcement learning for robots using neural networks*. PhD thesis, School of Computer Science, Carnegie Mellon University, 1993.

Maddison, C. J., Huang, A., Sutskever, I., and Silver, D. Move Evaluation in Go Using Deep Convolutional Neural Networks. In *ICLR*, 2015.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. Human-level control through deep reinforcement learning. *Nature*, 518 (7540):529–533, 2015.

Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., Maria, A. De, Panneershelvam, V., Suleyman, M., Beattie, C., Petersen, S., Legg, S., Mnih, V., Kavukcuoglu, K., and Silver, D. Massively parallel methods for deep reinforcement learning. In *Deep Learning Workshop, ICML*, 2015.

Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. In *ICLR*, 2016.

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. arXiv preprint arXiv:1506.02438, 2015.

Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 01 2016.

Simonyan, K., Vedaldi, A., and Zisserman, A. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.

Stadie, B. C., Levine, S., and Abbeel, P. Incentivizing exploration in reinforcement learning with deep predictive models. *arXiv preprint arXiv:1507.00814*, 2015.

Sutton, R. S. and Barto, A. G. *Introduction to reinforcement learning*. MIT Press, 1998.

Sutton, R. S., Mcallester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In *NIPS*, pp. 1057–1063, 2000.

van Hasselt, H. Double Q-learning. *NIPS*, 23:2613–2621, 2010.

van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double Q-learning. *arXiv preprint arXiv:1509.06461*, 2015.

van Seijen, H., van Hasselt, H., Whiteson, S., and Wiering, M. A theoretical and empirical analysis of Expected Sarsa. In *IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning*, pp. 177–184. 2009.

Watter, M., Springenberg, J. T., Boedecker, J., and Riedmiller, M. A. Embed to control: A locally linear latent dynamics model for control from raw images. In *NIPS*, 2015.

参考文献

Ba, J., Mnih, V., 和 Kavukcuoglu, K. 基于视觉注意力的多目标识别。载于*ICLR*，2015。Baird, L.C. 优势更新。技术报告 WL- TR-93-1146，赖特-帕特森空军基地，1993。Bellemare, M. G., Naddaf, Y., Veness, J., 和 Bowling, M. 街机学习环境：通用智能体的评估平台。人工智能研究期刊，47:253–279, 2013。

Bellemare, M. G., Ostrovski, G., Guez, A., Thomas, P. S., 和 Munos, R. 增大动作差距：强化学习的新算子。载于*AAAI*，2016。待发表。

Bengio, Y., Boulanger-Lewandowski, N., 和 Pascanu, R. 循环网络优化的进展。载于 *ICASSP*，第 8624–8628 页，2013。Fukushima, K. Neocognitron：一种用于不受位置偏移影响的模式识别机制的自组织神经网络模型。生物控制论，36: 193–202，1980。

Guo, X., Singh, S., Lee, H., Lewis, R. L., 和 Wang, X. 使用离线蒙特卡洛树搜索规划进行实时 Atari 游戏的深度学习。载于 *NIPS*，第 3338– 3346 页。2014。

Harmon, M.E. 和 Baird, L.C. 使用通用函数逼近的多玩家残差优势学习。技术报告 WL-TR-1065，赖特-帕特森空军基地，1996。Harmon, M.E., Baird, L.C., 和 Klopf, A.H. 应用于微分博弈的优势更新。载于 G. Tesauro, D.S. Touretzky 和 Leen, T.K.（编），*NIPS*，1995。

LeCun, Y., Bengio, Y., 和 Hinton, G. 深度学习。自然, 521(7553):436–444，2015。
Levine, S., Finn, C., Darrell, T., 和 Abbeel, P. 深度视觉运动策略的端到端训练。*arXiv* 预印本 *arXiv:1504.00702*, 2015. Lin, L.J. 使用神经网络的机器人强化学习。博士学位论文，卡内基梅隆大学计算机科学学院，1993. Maddison, C. J., Huang, A., Sutskever, I., 和 Silver, D. 使用深度卷积神经网络进行围棋落子评估。载于*ICLR*, 2015. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Ve- ness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wier- stra, D., Legg, S., 和 Hassabis, D. 通过深度强化学习实现人类水平的控制。自然, 518 (7540):529–533, 2015.

Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., Maria, A. De, Panneershelvam, V., Suley- man, M., Beattie, C., Petersen, S., Legg, S., Mnih, V., Kavukcuoglu, K., 和 Silver, D. 深度强化学习的大规模并行方法。载于深度学习研讨会，*ICML*, 2015. Schaul, T., Quan, J., Antonoglou, I., 和 Silver, D. 优先经验回放。载于 *ICLR*, 2016. Schulman, J., Moritz, P., Levine, S., Jordan, M. I., 和 Abbeel, P. 使用广义优势估计的高维连续控制。arXiv 预印本 arXiv:1506.02438, 2015. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., 和 Has- sabis, D. 利用深度神经网络和树搜索精通围棋。自然, 529(7587):484–489, 01 2016. Simonyan, K., Vedaldi, A., 和 Zisserman, A. 深入卷积网络内部：图像分类模型与显著性图的可视化。*arXiv* 预印本 *arXiv:1312.6034*, 2013. Stadie, B. C., Levine, S., 和 Abbeel, P. 利用深度预测模型激励强化学习中的探索。*arXiv* 预印本 *arXiv:1507.00814*, 2015. Sutton, R. S. 和 Barto, A. G. 强化学习导论。麻省理工学院出版社，1998. Sutton, R. S., Mcallester, D., Singh, S., 和 Mansour, Y. 使用函数逼近的强化学习策略梯度方法。载于*NIPS*，第 1057–1063 页, 2000. van Hasselt, H. 双重 Q 学习。*NIPS*, 23:2613–2621, 2010. van Hasselt, H., Guez, A., 和 Silver, D. 使用双重 Q 学习的深度强化学习。*arXiv* 预印本 *arXiv:1509.06461*, 2015. van Seijen, H., van Hasselt, H., Whiteson, S., 和 Wier- ing, M. Expected Sarsa 的理论与实证分析。载于 *IEEE* 自适应动态规划与强化学习研讨会，第 177–184 页。2009. Watter, M., Springenberg, J. T., Boedecker, J., 和 Ried- miller, M. A. 嵌入以实现控制：用于从原始图像进行控制的局部线性潜在动力学模型。载于 *NIPS*, 2015.